

Cross Validation in ML

Prof. Arun Chauhan, CSE
GEU, Dehradun

Cross-validation is a technique for validating the model efficiency by training it on the subset of input data and testing on previously unseen subset of the input data.

OR

It is a technique to check how a statistical model generalizes to an independent dataset.

Validation

The idea of validation procedures is to estimate E_{out} , using **a different dataset (validation set)** with respect to the dataset used for learning the model **(training set)**

Regularization and **validation** are two techniques that can (and must) be used together:

- **regularization** helps to learn a model that can generalize better
- **validation** gives an estimate of the out-of-sample error of the learnt model

Validation

The out-of-sample error can be seen as:

$$E_{\text{out}}(\boldsymbol{\theta}) = E_{\text{in}}(\boldsymbol{\theta}) + \text{penalty for model complexity}$$

Regularization

$$E_{\text{out}}(\boldsymbol{\theta}) = E_{\text{in}}(\boldsymbol{\theta}) + \underbrace{\text{penalty for model complexity}}$$

REGULARIZATION estimates this quantity

Validation

$$\underbrace{E_{\text{out}}(\boldsymbol{\theta})}_{\text{Validation}} = E_{\text{in}}(\boldsymbol{\theta}) + \text{penalty for model complexity}$$

VALIDATION estimates this quantity

Validation set

The idea of a **validation** set is to estimate the model's **performance out of sample**. A common procedure is

1. **Remove** a subset from the training data → this subset is not used for training
2. **Train** the model on the remaining training data → the model will be trained on less data
3. **Evaluate** the model performance on the **held-out** set → this is an unbiased estimation of the out-of-sample error
4. **Retrain** the model on **all** the data

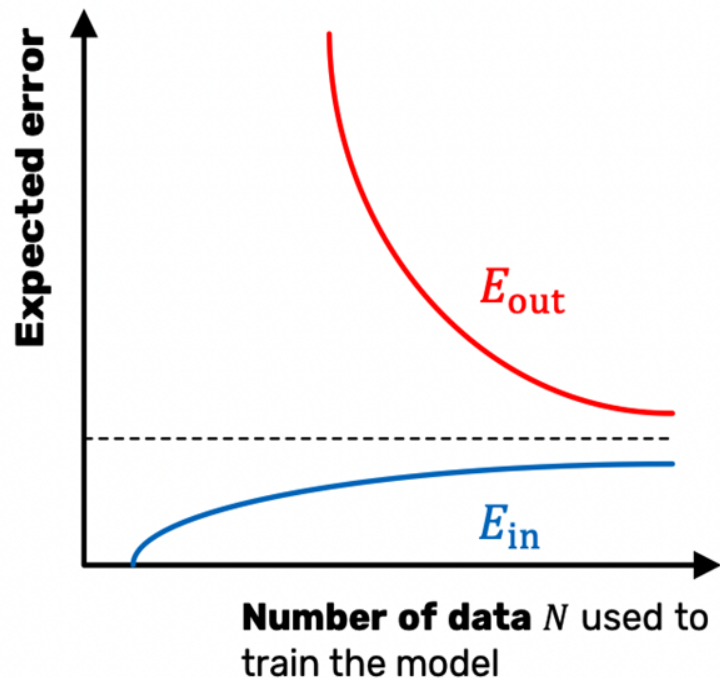
Validation set

Suppose to have the dataset $\mathcal{D} = \{(\mathbf{x}(1), y(1)), \dots, (\mathbf{x}(N), y(N))\}$. We proceed as follows:

N_{val} data: **validation**
 $\mathcal{D}_{\text{val}}$

$N - N_{\text{val}}$ data: **training**
 $\mathcal{D}_{\text{train}}$

- N_{val} **«small»**: bad estimate of E_{out}
- N_{val} **«big»**: possibility to learn a bad model (look at learning curves)



Validation set

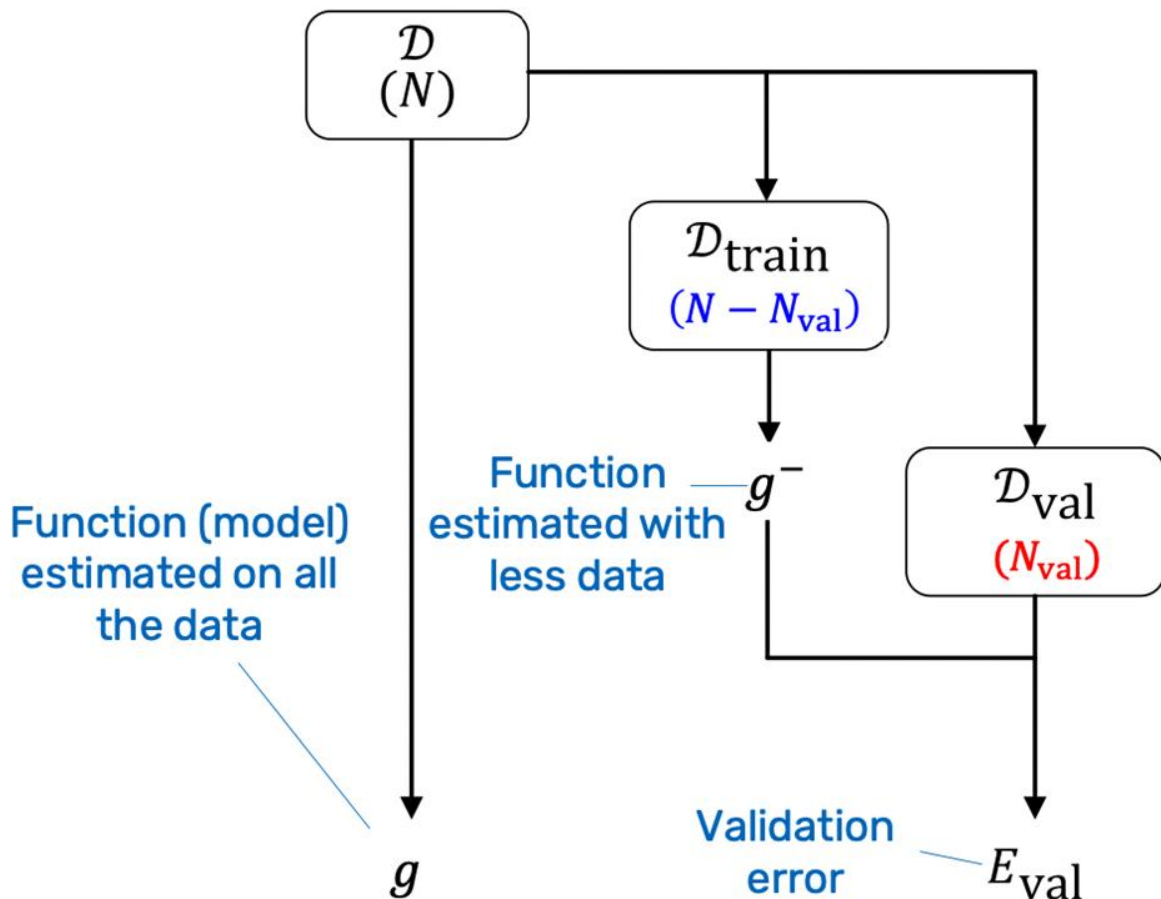
$$\begin{array}{ccccc} \mathcal{D} & \rightarrow & \mathcal{D}_{\text{train}} & \cup & \mathcal{D}_{\text{val}} \\ \downarrow & & \downarrow & & \downarrow \\ N & & N - N_{\text{val}} & & N_{\text{val}} \end{array}$$

$$\mathcal{D} \Rightarrow g \quad \mathcal{D}_{\text{train}} \Rightarrow g^{-}$$

$$E_{\text{val}} = E_{\text{val}}(g^{-})$$

Rule of thumb

$$N_{\text{val}} = \frac{N}{5}$$



Model selection using validation

Validation procedures can be used for two purposes:

1. **Evaluate the performance** of the estimated model (e.g. estimate E_{out})
2. **Choose the best model** from a set of different models

For instance, choosing the best model includes:

- Choose between a linear and a nonlinear model
- Choose the number of features\regressors to use
- Choose the value of the regularization hyperparameter λ_{reg}
- ... any other choice that affects the model's learning procedure

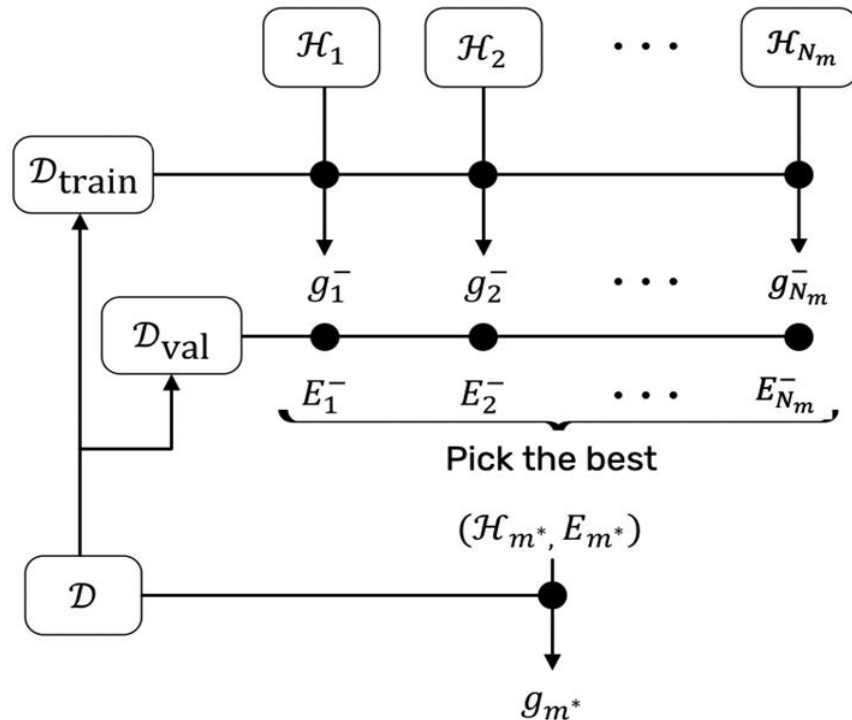
Model selection using validation

Assume to have N_m **models set** $\mathcal{H}_1, \mathcal{H}_2, \dots, \mathcal{H}_{N_m}$
from which to learn the model

- **Learn** g_m^- using $\mathcal{D}_{\text{train}}$ for each models set
- **Evaluate** g_m^- using $\mathcal{D}_{\text{val}}$

$$E_m^- = E_{\text{val}}(g_m^-) \quad m = 1, \dots, N_m$$

- **Select** the model $m = m^*$ with **lowest** error E_m^-



Model selection using validation

Problem: if we use the validation dataset $\mathcal{D}_{\text{val}}$ «many times» to make choices (e.g. choosing between different models), then the validation dataset $\mathcal{D}_{\text{val}}$ **no longer provides a good estimate of the out-of sample error** E_{out}

Intuition: use $\mathcal{D}_{\text{val}}$ to make choices about which model to use makes it possible that **such choices are dependent on the particular data values** contained in $\mathcal{D}_{\text{val}}$. Who guarantees us that with different data we would have made the same choices?

What is happening is that we are **overfitting the validation set**

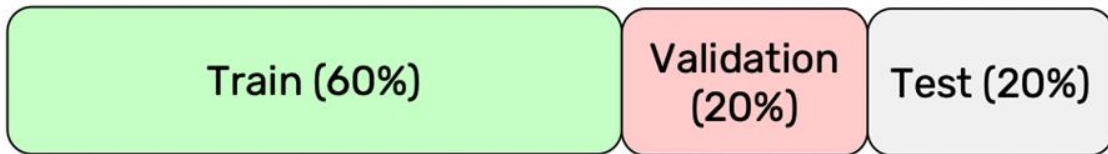
Solution: there is the need of a 3rd dataset. **The test dataset, on which we will compute the test error** E_{test}

Dataset «contamination»

We have until now obtained **three estimates** of the error E_{out}

Contamination: optimistic bias in E_{out} estimate (e.g. tell that E_{out} is smaller than it actually is)

- **Training set:** totally contaminated
- **Validation set:** slightly contaminated
- **Test set:** totally «clean»



Cross-validation

Dividing the dataset in **three parts** (train, validation, test) is feasible if the available data are many (where «many» depends on the problem ... look at the learning curves)

We would like that:

$$E_{\text{out}}(g) \approx E_{\text{out}}(g^-) \approx E_{\text{val}}(g^-)$$

(N_{val} small) (N_{val} big)

It's the only one I can compute!

- **Big** N_{val} : in this case, the value of E_{val} computed using g^- would be similar to the value of E_{out} computed from g^- , since we use many data N_{val} for the validation. Recall that the aim of computing E_{val} is to estimate E_{out}
- **Small** N_{val} : in this case, the value of E_{out} computed using g^- would be similar to the value E_{out} computed from g (the function estimated on all the data), since we use many data $N - N_{\text{val}}$ for training g^- . This is the value that interests us but that we cannot compute directly

Cross-validation

Cross-validation allows to «have N_{val} both large and small»

Leave-one-out cross-validation

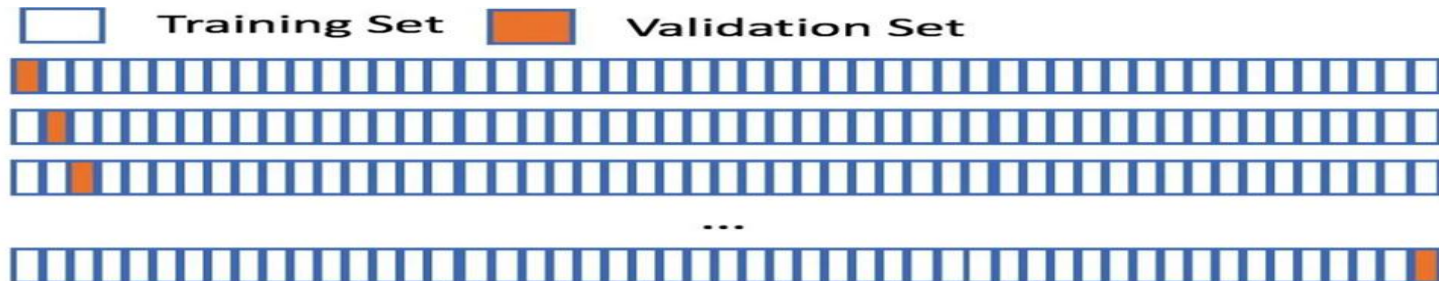
Use $N - 1$ data for training and $N_{\text{val}} = 1$ datum for validation

$$\mathcal{D}_i = \{\mathbf{x}(1), y(1)\}, \dots, \{\mathbf{x}(i), y(i)\}, \dots, \{\mathbf{x}(N), y(N)\}$$

- Datum removed from training set and used for validation

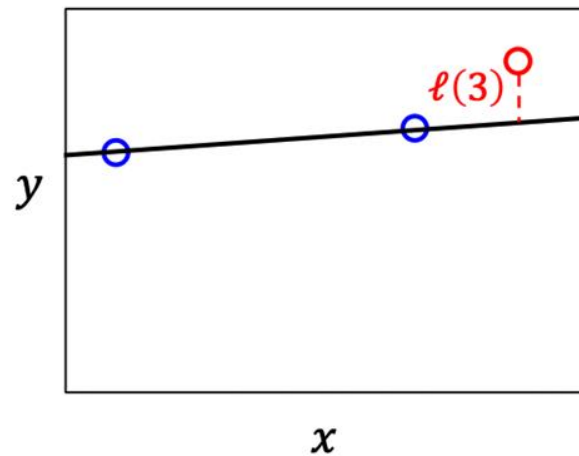
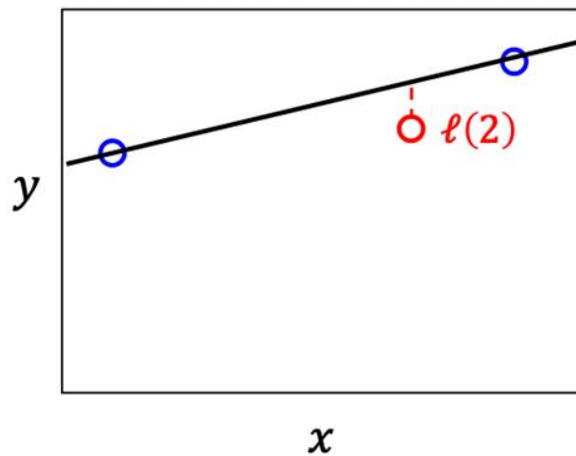
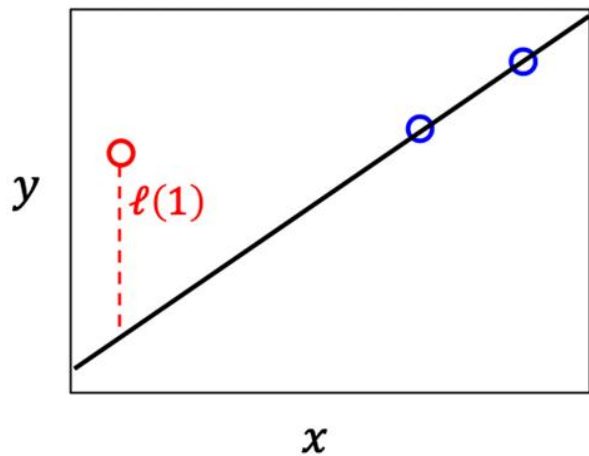
where $\mathcal{D}_i$ is the dataset without the i -th datum.

The function (model) learned using $\mathcal{D}_i$ is denoted as g_i^-



Cross-validation example

Example: we want to learn a linear model using a regressor and $N = 3$ observations, using $N_{\text{val}} = 1$ point to do cross-validation



$$E_{\text{cv}} = \frac{1}{3} (\ell(1) + \ell(2) + \ell(3))$$

Cross-validation

The validation error on the «held-out» point $x(i)$ is $\ell(i) = E_{\text{val}}(g_i^-) = \ell(y(i), g_i^-(\varphi(i)))$

It is possible to define the **cross-validation error** E_{cv} as:

$$E_{\text{cv}} = \frac{1}{N} \sum_{i=1}^N \ell(i)$$

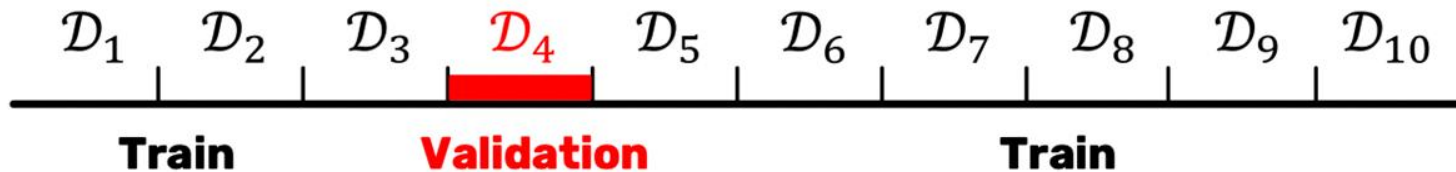
- In this way, we estimate N models using $N - 1$ data, and we validate them using N estimates of the validation error (each computed on $N_{\text{val}} = 1$ data)
- It is also possible to compute the standard (sample) deviation of the errors $\ell(i)$. If it is large, it means that the model is very sensitive to the data it is being trained on

***K*-fold cross-validation**

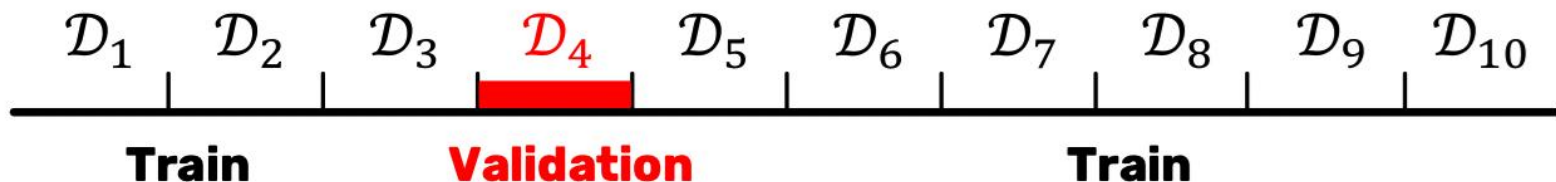
Cross-validation with $N_{\text{val}} = 1$ (leave-one-out cross-validation) has disadvantages like:

- It is **computationally expensive**. In case we would like to use it to choose between M models, it would require a total of N training sessions for each of the M models
- The error estimate E_{cv} has a **high variance**, since it is based on only one datum

It is possible to reserve more points for the validation by dividing the training set in «**folds**». For instance, if $K = 10$ we would have



***K*-fold cross-validation**



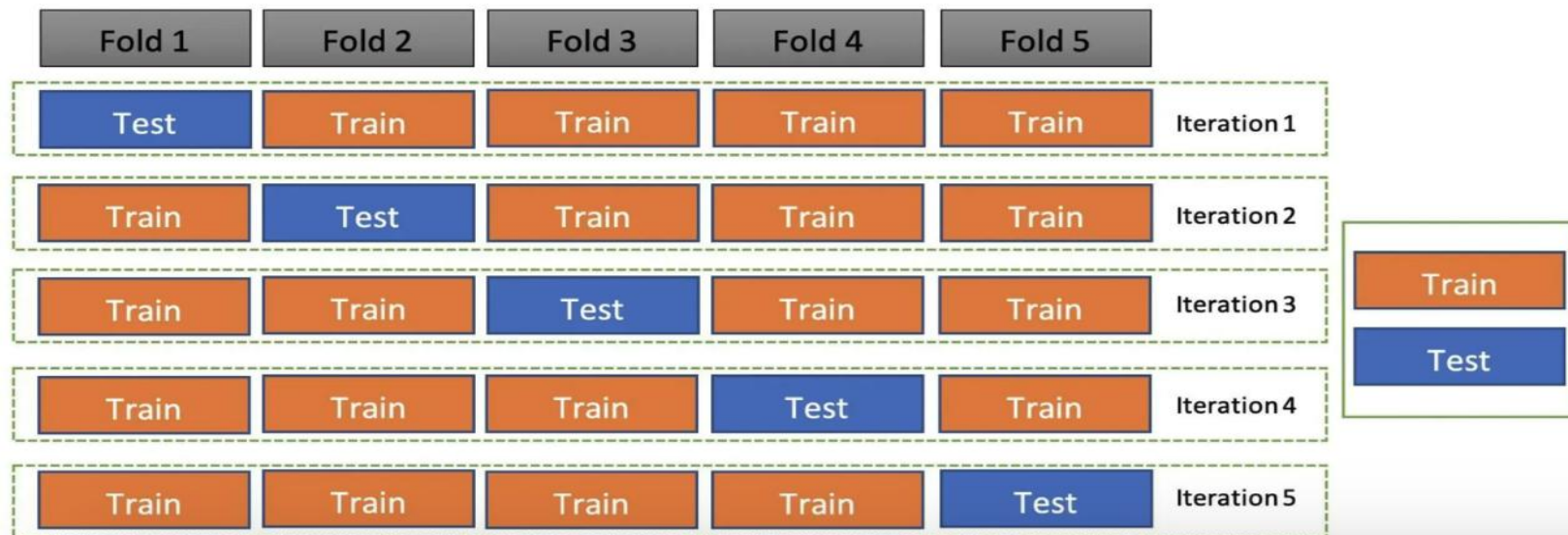
- *K*-fold cross-validation requires N/N_{val} training sessions, each with $N - N_{\text{val}}$ data
- A good **tradeoff** is to use $K = 10$

$$\text{10-fold cross validation: } N_{\text{val}} = \frac{N}{10}$$

K-fold cross-validation

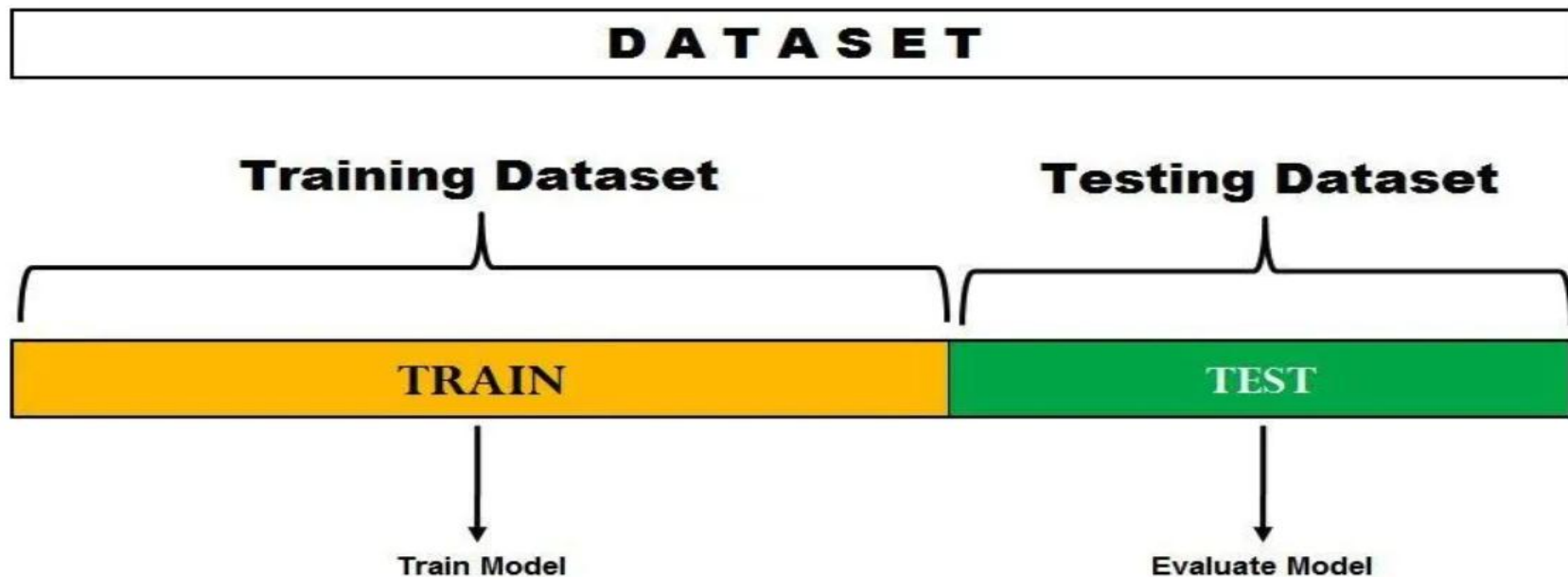
In this technique, the whole dataset is partitioned in k parts of equal size and each partition is called a fold. It's known as k -fold since there are k parts where k can be any integer - 3,4,5, etc.

One fold is used for validation and other $K-1$ folds are used for training the model. To use every fold as a validation set and other left-outs as a training set, this technique is repeated k times until each fold is used once.



Holdout cross-validation

Also called a train-test split, holdout cross-validation has the entire dataset partitioned randomly into a training set and a validation set. A rule of thumb to partition data is that nearly 70% of the whole dataset will be used as a training set and the remaining 30% will be used as a validation set. Since the dataset is split into only two sets, the model is built just one time on the training set and executed faster.



D A T A S E T

Training Dataset

Validation Dataset

Testing Dataset

TRAIN

VALIDATION

TEST

Train multiple Models

(e.g. Logistic Regression,
Decision Trees, KNN)

Validate Models

Tune Hyper parameters and
Select the Best Model
(e.g. Logistic Regression)

Evaluate Model

Evaluate the model based on various
metrics
(e.g. Confusion Matrix to evaluate the final
performance of the selected Logistic
Regression Model)

Bootstrapping

A resampling technique that estimates the sampling distribution of a statistic by drawing **with replacement** from the original data.

Core Idea

Given a data set of size N , create many new bootstrap samples of size N by sampling with replacement. Compute the statistic (or train the model) on each sample. The collection of B results approximates the true sampling distribution of that statistic.

How it works

- Start with original data set of size N
- Draw a new sample of size N with replacement
- Compute the quantity of interest (model, error, accuracy...)
- Repeat B times (typically $B = 100 - 1000+$)
- Use the empirical distribution of the B results

Why we use it

- Estimate variance & confidence intervals without strong parametric assumptions
- Assess model stability / sensitivity to the particular training set
- Approximate the distribution of almost any statistic
- Useful when data are limited and closed-form formulas are unavailable

Relation to cross-validation: Both are resampling methods. CV systematically holds data out; bootstrap resamples with replacement and points can appear multiple times.

Thanks